Anderson Contreira
Posted on Nov 28, 2025

Edit Manage Stats

RESTful – Pílula 5 – Versionamento de APIs RESTful

#braziliandevs #rest #code #restapi

Com o tempo, qualquer API evolui.
Novos campos surgem, regras mudam, recursos são removidos e estruturas são reorganizadas.
Sem versionamento, qualquer mudança pode quebrar clientes que dependem daquele contrato.

Por isso, **versionar APIs é essencial** para garantir evolução sem interromper quem já integra com você.

Por que versionar uma API?

Porque APIs mudam — e quando mudam, você precisa garantir:

- compatibilidade entre clientes antigos e novos
- segurança para introduzir breaking changes
- um caminho de migração previsível

- documentação clara, separada por versão
- estabilidade durante o ciclo de vida da API

Formas de versionamento

1️⃣ Versionamento na URL (o mais comum e explícito)

```
GET /v1/users
GET /v2/users
```

Simples, direto e fácil de documentar.

2️⃣ Versionamento por Header

Via `Accept Header`:

```
Accept: application/vnd.myapp.v1+json
```

Manter as URLs limpas é um benefício, mas exige maturidade maior dos clientes.

> Nota: É uma opção válida, mas eu recomendo usar o caminho mais tradicional e simples que é via URL.

3️⃣ Versionamento via Query Parameter (não recomendado)

```
GET /users?version=2
```

Evite — confunde cache, roteamento e não é considerado RESTful.

> Nota: Em resumo, não use!!!

Arquitetura: como manter múltiplas versões?

Dependendo do design e da maturidade do sistema, duas abordagens são comuns:

Abordagem 1 — Versões dentro da mesma aplicação

A mesma aplicação (ou monólito, ou service) contém:

- código da versão 1
- código da versão 2
- rotas /v1 e /v2 expostas lado a lado

Esse modelo é muito comum e simples de manter quando as versões compartilham 80% da base lógica.

Prós:

- facilidade de deploy
- menos repositórios
- menor custo infra inicial

Contras:

- código mais complexo
- necessidade de controle de legado dentro do mesmo código
- risco de acoplamento entre versões

Abordagem 2 — Versões como aplicações totalmente separadas

Cada versão é uma aplicação distinta, com seu próprio repositório:

- `api-v1` (repo separado)
- `api-v2` (repo separado)

Essa abordagem é comum quando:

- V2 é radicalmente diferente da V1
- há reescrita completa
- existe intenção de isolar responsabilidades

Prós:

- maior isolamento
- codebase mais limpa
- ciclo de vida independente

Contras:

- mais repositórios
- mais custos operacionais
- mais pipelines e infraestrutura

📌 *Nota:* Essa parte arquitetural será explorada em um **artigo técnico dedicado**.

Quando realmente criar uma nova versão?

Crie uma nova versão quando houver **breaking changes**, como:

- mudança estrutural de payloads
- renomeação ou remoção de campos
- mudança na semântica dos recursos
- alteração no comportamento de endpoints
- mudanças que impactem clientes antigos

Adicionar novos campos **não exige versão nova**.

Versionamento também pode acontecer no API Gateway

Além do versionamento na aplicação, Gateways também podem cuidar disso.
É muito usado em arquiteturas de médio e grande porte.

Kong API Gateway

No Kong, é possível:

- criar **routes** separadas para cada versão
- apontar `/v1/*` para um upstream
- apontar `/v2/*` para outro serviço
- controlar ciclo de vida de versões facilmente

Exemplo:

- `/v1/` → `api-v1-service`
- `/v2/` → `api-v2-service`

Também permite ativar/desativar versões sem alterar a aplicação.

AWS API Gateway

No API Gateway da AWS, você pode:

- usar **stages** para cada versão (`v1`, `v2`, `v3`)
- mapear rotas específicas para Lambdas ou serviços distintos
- versionar por **deploy stage + base path mapping**
- ter controle fino de throttling e lifecycle por versão

É excelente para arquiteturas serverless ou híbridas.

GCP API Gateway

O API Gateway do Google permite:

- criar configurações separadas de API para cada versão
- publicar múltiplas versões lado a lado
- usar **OpenAPI specs** distintas para cada release
- rotear `/v1/` e `/v2/` para backends diferentes

Simples e direto, com boa integração ao GCP.

Exemplo prático de versionamento

Versão 1

Endpoint:

```
GET /v1/products/10
```

Resposta:

```
{
  "id": 10,
  "name": "Camiseta",
  "price": 39.90
}
```

Versão 2 (mudança estrutural)

```
GET /v2/products/10
```

Resposta:

```
{
  "id": 10,
  "title": "Camiseta",
  "pricing": {
    "amount": 39.90,
    "currency": "BRL"
  }
}
```

Resumo

- Versionamento é essencial para estabilidade e evolução
- O modelo mais comum é `/v1/`
- Headers podem ser uma alternativa
- Query params não são recomendados (Não faça isso)
- Você pode versionar pela aplicação **ou** pelo API Gateway

- Versionamento pode significar uma app com múltiplas versões **ou** apps separadas
- Crie versões novas **somente** quando houver breaking changes

Top comments (0)

Code of Conduct • Report abuse

Anderson Contreira

Solutions Architect. IT professional since 2008. Over 6 years of leadership experience. Specialist in distributed architecture, microservices, cloud (AWS/GCP), DevOps and Full-Stack experience.

LOCATION
Brazil

EDUCATION
Distributed Software Architecture @ PUC Minas

WORK
Cloud Software Archtect @ Caylent

JOINED
Oct 31, 2025

More from Anderson Contreira

ALERTA IMPORTANTE PARA DEVS: golpe em testes técnicos está crescendo - fiquem espertos
#braziliandevs #scam #recruiting #development

Planejamento de carreira: ou você assume o volante, ou vira passageiro da própria vida
#braziliandevs #career

RESTful – Pílula 6 – Paginação, Filtros e Ordenação em APIs REST
#braziliandevs #rest